

Cryptanalyse et factorisation du RSA pour décrypter
un message électronique

Mise en œuvre

Guide d'utilisateur

YAHIAOUI Hadj Habib

Chaos.hh@gmail.com

Hadj-habib.yahiaoui@sonatrach.dz

Tél : +213661513881

Mise en œuvre

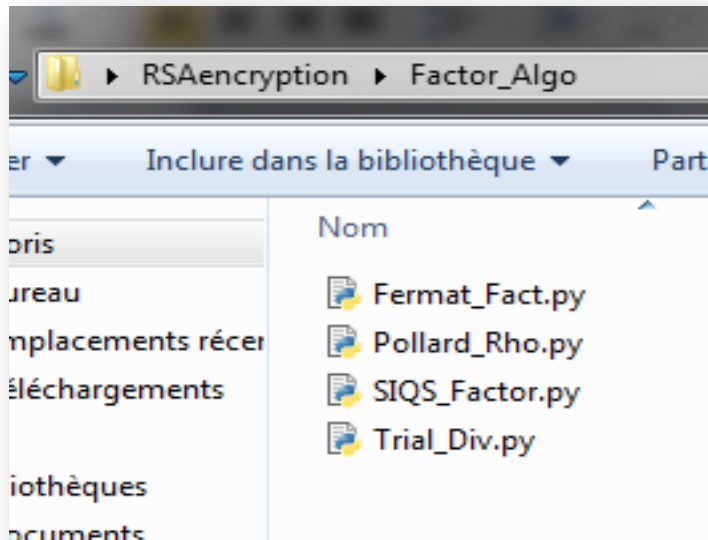
Guide d'utilisateur

Il s'agit d'un simple guide d'utilisation de la plateforme réalisée et des scripts développés dans le cadre de la PGS en sécurité des systèmes d'informations 2018/2019. Le thème de cette recherche est « Cryptanalyse et factorisation du RSA pour décrypter un message électronique ». La mise en œuvre est constituée de deux parties principales : la factorisation entière en facteurs premiers ; un ensemble de méthodes, chaque méthode est implémentée dans un script distinct, et l'attaque du cryptosystème RSA ; une plateforme qui se compose d'un cryptosystème RSA complet que nous avons développé dans le contexte de cette étude, des scripts de cryptanalyse qui aident à retrouver la clé privée pour au final, déchiffrer un message crypté et intercepté (détourné) en utilisant l'attaque MITM. Le langage utilisé est le Python avec sa version 3.8.

1- Factorisation entière en facteurs premiers :

Dans cette partie, nous avons implémenté quatre algorithmes de factorisation appliqués sur différents modulus RSA de différentes tailles.

Dans le dossier principal **RSAencryption**, on trouve le sous-dossier **Factor_Algo** qui contient les quatre scripts des quatre méthodes de factorisation comme le montre la figure suivante :



On doit installer préalablement le Python 3.8, aucune bibliothèque externe n'est nécessaire.

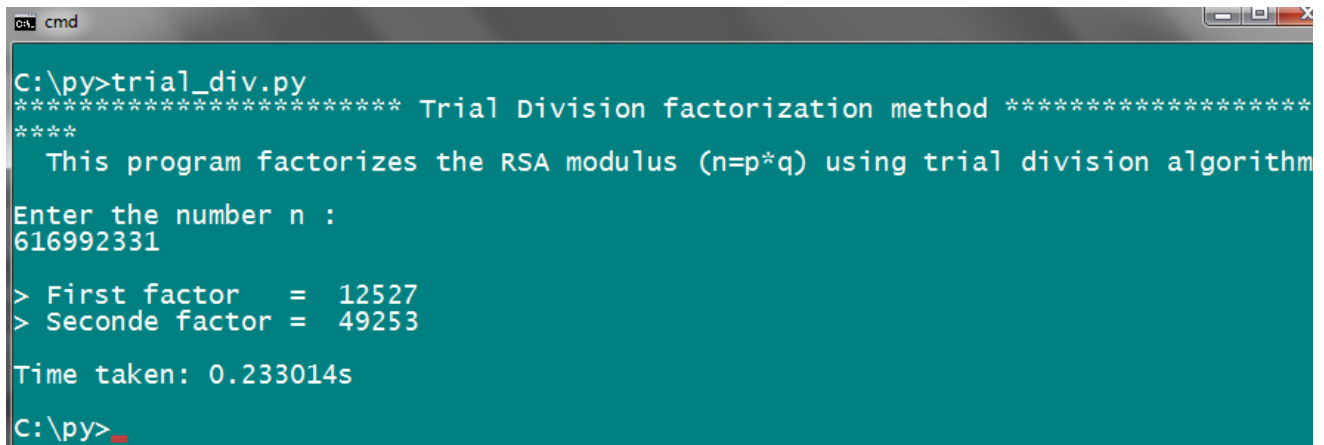
Pour exécuter ces programmes, on va lancer l'invite de commande, et se positionner sur le dossier en question, les figures suivantes montrent un exemple d'utilisation, on commençant par les divisions successives avec le script **trial_div.py** :

```
cmd - trial_div.py

C:\py>trial_div.py
***** Trial Division factorization method *****
****
This program factorizes the RSA modulus (n=p*q) using trial division algorithm
Enter the number n :

```

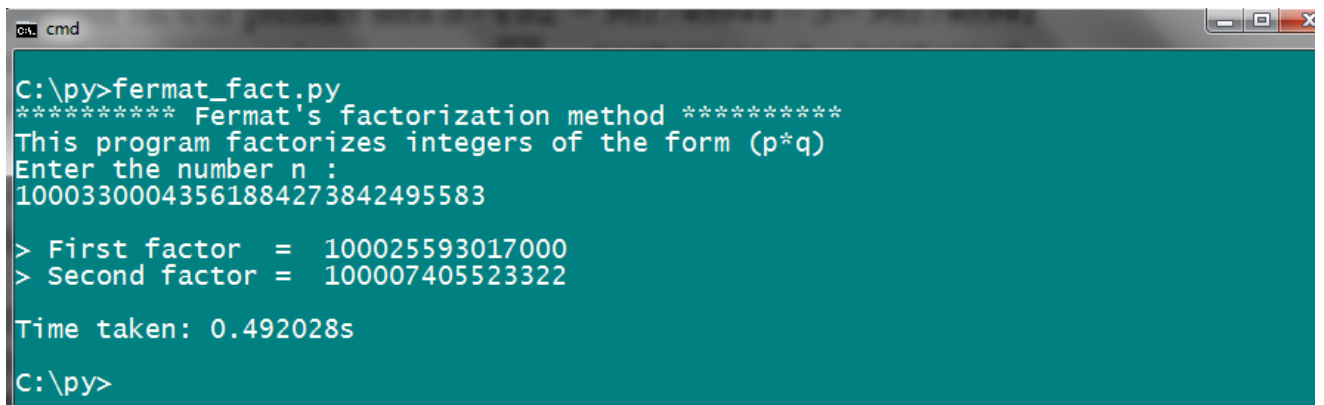
Il faut introduire le nombre à factoriser (Modulus) et valider :



```
cmd
C:\py>trial_div.py
***** Trial Division factorization method *****
***
  This program factorizes the RSA modulus (n=p*q) using trial division algorithm
Enter the number n :
616992331
> First factor    = 12527
> Seconde factor = 49253
Time taken: 0.233014s
C:\py>
```

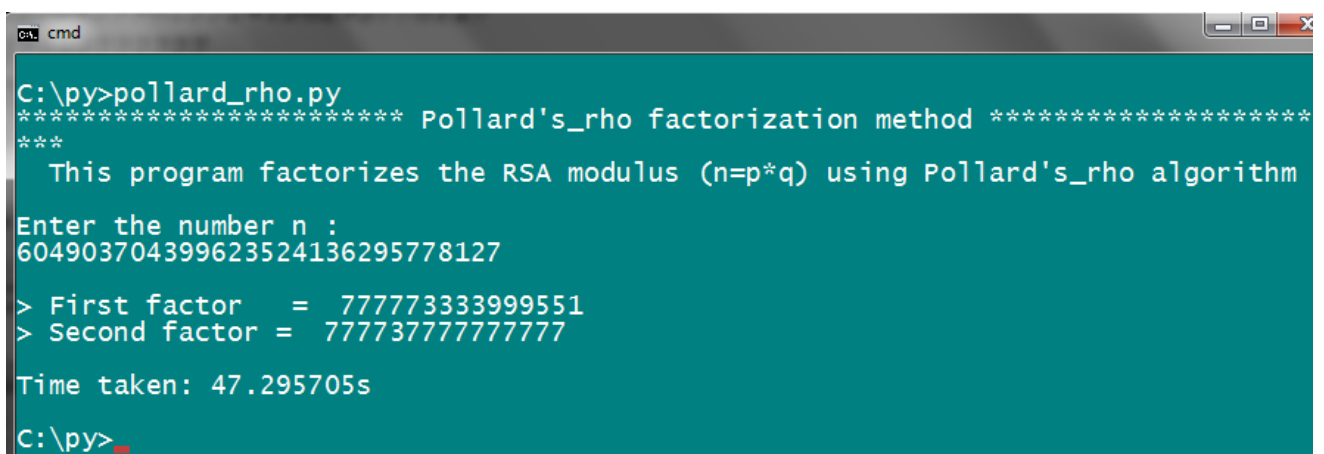
Et voilà, notre modulus est factorisé en 0.233014 secondes.

La même chose pour les autres méthodes, la figure suivante montre l'exécution du programme **Fermat_Fact.py** pour la méthode de Fermat :



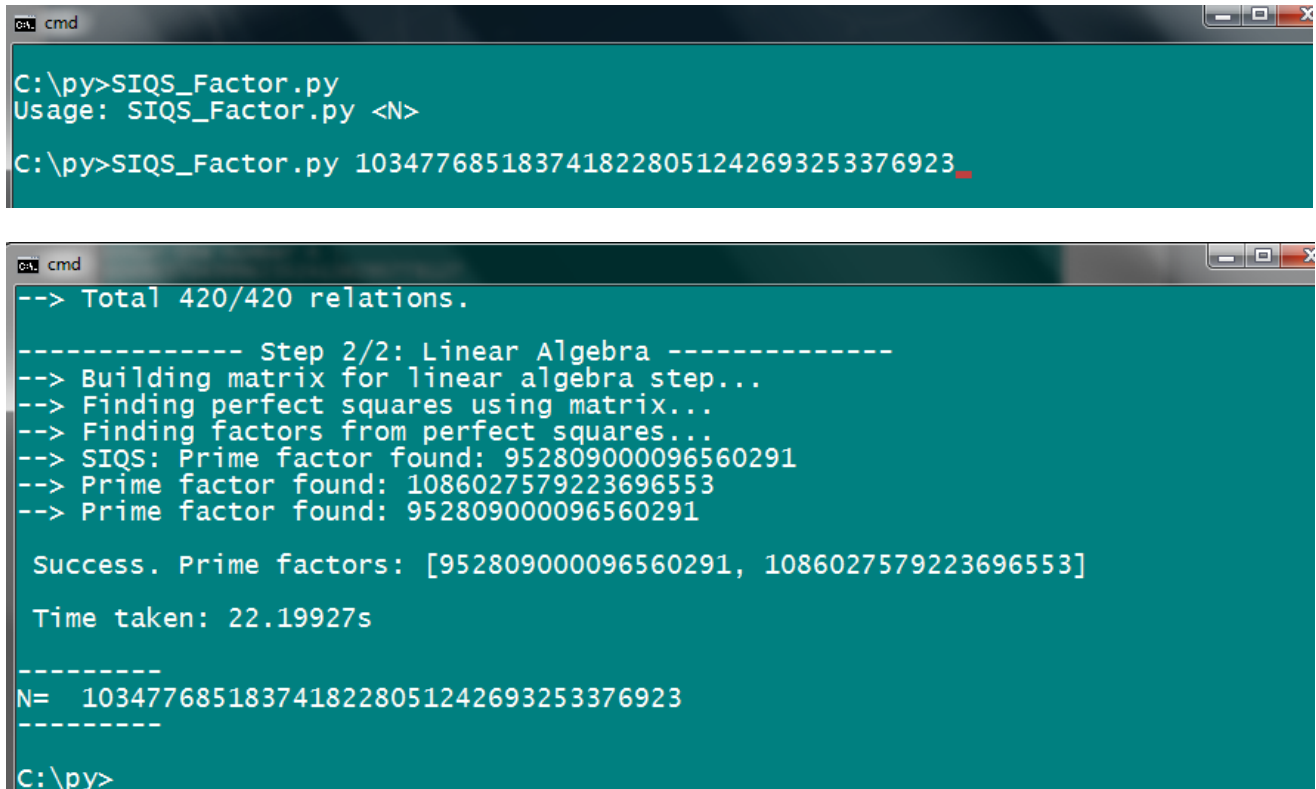
```
cmd
C:\py>fermat_fact.py
***** Fermat's factorization method *****
This program factorizes integers of the form (p*q)
Enter the number n :
10003300043561884273842495583
> First factor    = 100025593017000
> Second factor   = 100007405523322
Time taken: 0.492028s
C:\py>
```

Et pour la méthode Rho de pollard, on utilise le script **pollard_rho.py** :



```
cmd
C:\py>pollard_rho.py
***** Pollard's_rho factorization method *****
***
  This program factorizes the RSA modulus (n=p*q) using Pollard's_rho algorithm
Enter the number n :
604903704399623524136295778127
> First factor    = 777773333999551
> Second factor   = 77773777777777
Time taken: 47.295705s
C:\py>
```

Concernant la dernière méthode, le crible quadratique, on doit introduire le modulus à factoriser comme paramètre comme le montre la figure suivante avec le script **SIQS_Factor.py** :



```
cmd
C:\py>SIQS_Factor.py
Usage: SIQS_Factor.py <N>

C:\py>SIQS_Factor.py 1034776851837418228051242693253376923_

cmd
--> Total 420/420 relations.

----- Step 2/2: Linear Algebra -----
--> Building matrix for linear algebra step...
--> Finding perfect squares using matrix...
--> Finding factors from perfect squares...
--> SIQS: Prime factor found: 952809000096560291
--> Prime factor found: 1086027579223696553
--> Prime factor found: 952809000096560291

Success. Prime factors: [952809000096560291, 1086027579223696553]

Time taken: 22.19927s

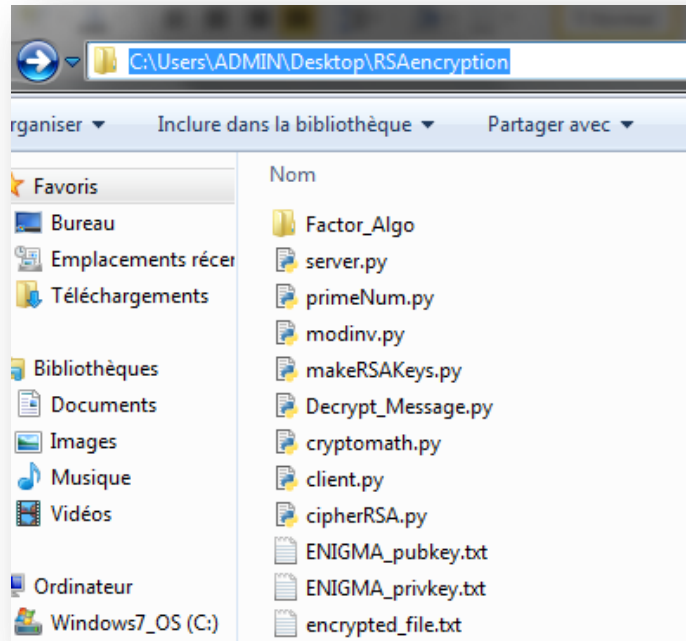
-----
N= 1034776851837418228051242693253376923
-----

C:\py>
```

2- Attaque du cryptosystème RSA

La plateforme se compose de trois machines, une machine pour le client, une machine pour le serveur et une machine pour le cryptanalyste. Dans cet exemple, nous avons utilisé deux machines linux (Lubuntu) pour le client et le serveur et une machine Windows 7 pour le cryptanalyste (interception des messages et cryptanalyse).

Pour la configuration de l'environnement, on doit avoir trois machines dans le même réseau (LAN), les scripts utilisés sont dans la racine du dossier RSAencryption, la figure suivante montre le dossier en question :

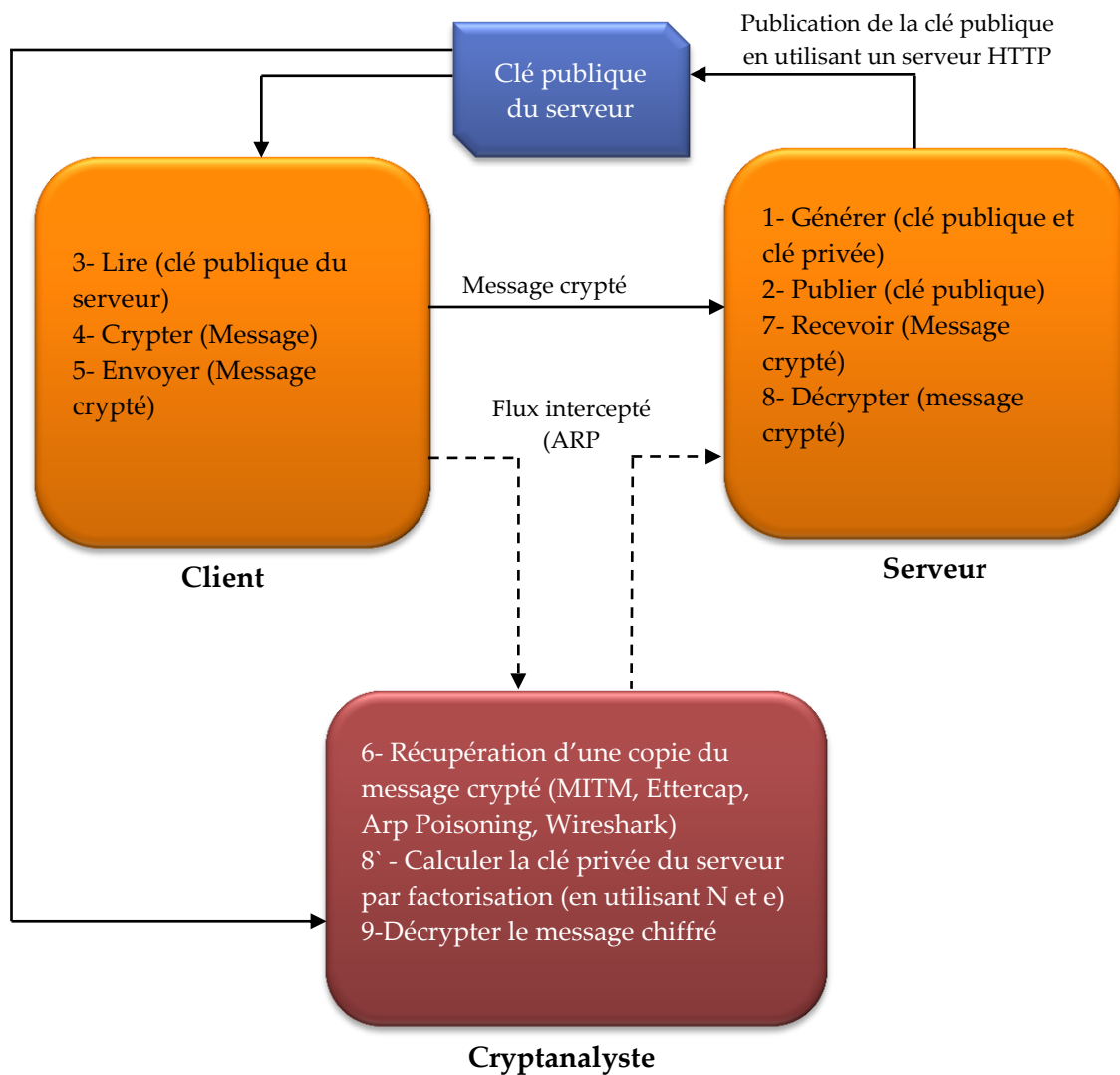


Parmi ces scripts, on a des programmes (modules) distincts et indépendants, chaque programme est responsable d'une tâche précise dans le processus de cette partie, ces programmes sont : **server.py** pour le programme serveur qui va initier la communication, **client.py** pour le client et qui va créer une session de communication avec le serveur, **makeRSAKeys.py** qui permet de générer les paires de clés (publiques et privés) selon la taille voulu (en bits), **modinv.py** qui permet de calculer l'exposant de déchiffrement **d** en utilisant l'inverse modulaire et le dernier programme est **Decrypt_Message.py** qui permet de décrypter le message chiffré et intercepté en utilisant la clé privée reconstruite. Et on a aussi des scripts qui contiennent des fonctions de calcul mathématique qu'on va les utiliser en tant que bibliothèques comme le script **cipherRSA.py** qui contient la routine complète du chiffrement utilisée dans le programme **client.py** et qui permet de chiffré les messages avant leurs envoi.

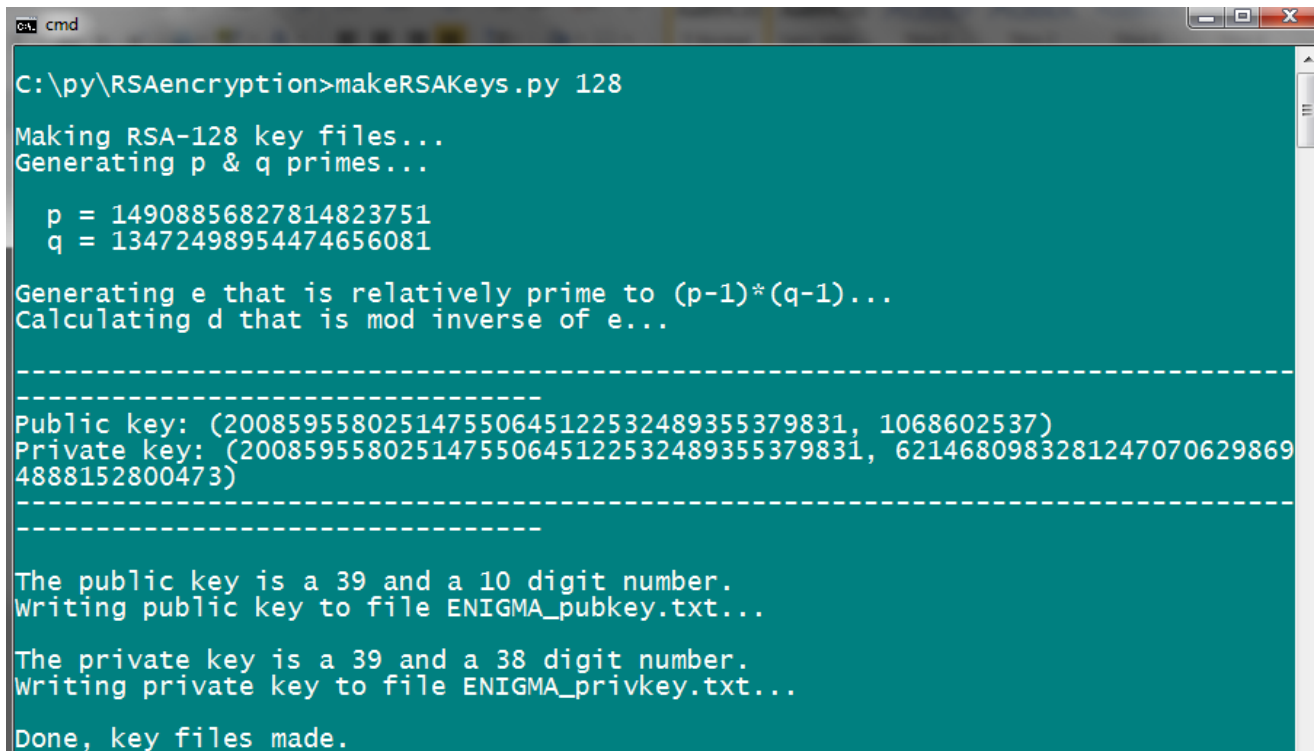
On trouve aussi les fichiers textes suivants:

- **ENIGMA_pubkey.txt** : contient la clé publique générée
- **ENIGMA_privkey.txt** : contient la clé privée générée
- **encrypted_file.txt** : contient le message chiffré intercepté dans la partie cryptanalyse

Le schéma de la plateforme est montré dans le diagramme suivant :



Selon ce diagramme, on commence par générer la paire des clés RSA (publique et privée) du serveur sur la première machine en utilisant le script **MakeRSAKeys.py** développé dans le cadre de cette étude :



```
C:\py\RSAencryption>makeRSAKeys.py 128
Making RSA-128 key files...
Generating p & q primes...

p = 14908856827814823751
q = 13472498954474656081

Generating e that is relatively prime to (p-1)*(q-1)...
Calculating d that is mod inverse of e...

-----
Public key: (200859558025147550645122532489355379831, 1068602537)
Private key: (200859558025147550645122532489355379831, 62146809832812470706298694888152800473)
-----

The public key is a 39 and a 10 digit number.
Writing public key to file ENIGMA_pubkey.txt...

The private key is a 39 and a 38 digit number.
Writing private key to file ENIGMA_privkey.txt...

Done, key files made.
```

Dans cette figure, on peut voir la génération de la paire des clés (publique et privée) (128 bits) en générant aléatoirement deux nombres premiers p et q .

- $p = 14908856827814823751$
- $q = 13472498954474656081$
- Le modulus $N = 200859558025147550645122532489355379831$
- L'exposant de chiffrement $e = 1068602537$
- L'exposant de déchiffrement $d = 62146809832812470706298694888152800473$

On va par la suite publier la clé publique en utilisant le serveur web « `http.server` » écrit en Python est préinstallé sur Lubuntu, une fois la clé publiée, on peut facilement la récupérer depuis n'importe quelle machine en utilisant un simple navigateur web.

Dans la machine du serveur, on lance le programme de communication serveur **server.py** en mentionnant l'adresse IP locale et le numéro de port à utiliser (un port libre) comme le montre la figure suivante :

```

C:\py\RSAencryption>server.py
-----
                Secret Communication Server
-----

Enter host/ip and Port:
Host: 192.168.243.1
Port: 33333

Creating new server socket ( 192.168.243.1 / 33333 ) ...
Creating new thread to wait for connections ...
-----
  
```

On peut constater que le programme serveur a créé le socket (192.168.243.1/33333) avec succès, et le thread listener est exécuté en arrière-plan afin d'intercepter toute nouvelle connexion.

Dans la deuxième machine, on va télécharger en premier lieu la clé publique du serveur, et, on va lancer la partie client du programme de communication en introduisant les informations du serveur :

```

                Secret Communication Server
-----

Enter host/ip and Port:
Host: 192.168.243.1
Port: 33333

Creating new server socket ( 192.168.243.1 / 33333 ) ...
Creating new thread to wait for connections ...
-----
New connection at ID 0 ('192.168.243.1', 57957)
-----

C:\Windows\System32>cd\
C:\>cd py
C:\py>cd rsaencryption
C:\py\RSAencryption>client.py
-----
                Secret Communication Client (RSA Encryption)
-----

Enter host/ip and Port for the communication server:
Host: 192.168.243.1
Port: 33333
Connected to server
-----
  
```

Une fois la connexion établie on peut commencer l'envoi des messages chiffrés :

```

cmd - server.py
Secret Communication Server
-----
Enter host/ip and Port:
Host: 192.168.243.1
Port: 33333
Creating new server socket ( 192.168.243.1 / 33333 ) ...
Creating new thread to wait for connections ...
-----
New connection at ID 0 ('192.168.243.1', 57957)
ID 0: 4_20_105166934018659565548925476
ID 0: 306837582976

cmd - client.py
C:\py\RSAencryption>client.py
Secret Communication Client (RSA Encryption)
-----
Enter host/ip and Port for the communication server:
Host: 192.168.243.1
Port: 33333
Connected to server
-----
test
  
```

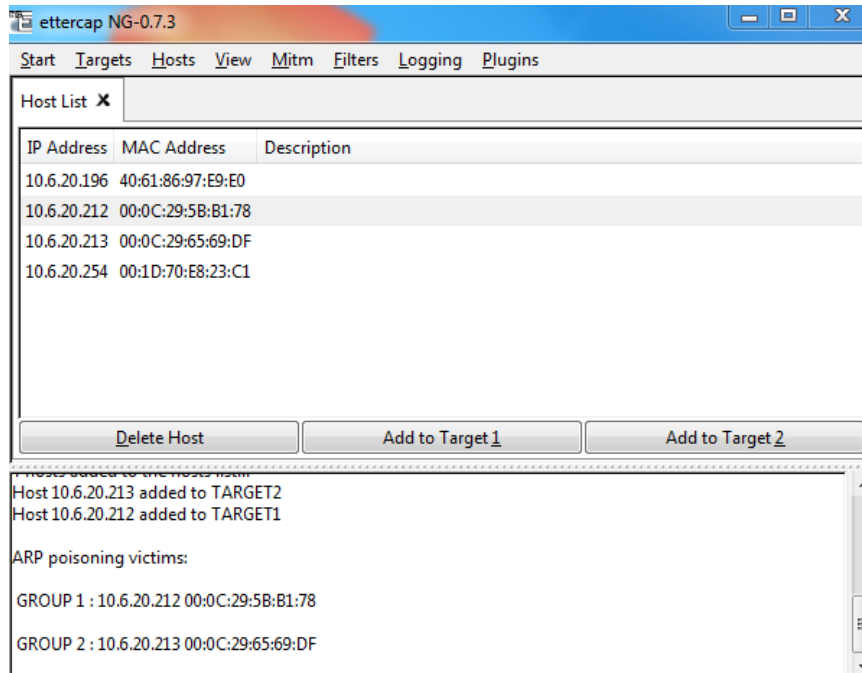
On peut facilement constater l'établissement d'une communication chiffrée en recevant le message crypté : 4_20_105166934018659565548925476 306837582976 dans la console du serveur.

Et pour commencer la partie de la cryptanalyse, on doit trouver un moyen de recevoir le message chiffré avant qu'il soit délivré au client, pour cela, on va mettre en œuvre une attaque très connu, qui est l'attaque man-in-the-middle (MITM) en utilisant l'empoisonnement du cache ARP via **ettercap** (voir l'encadré).

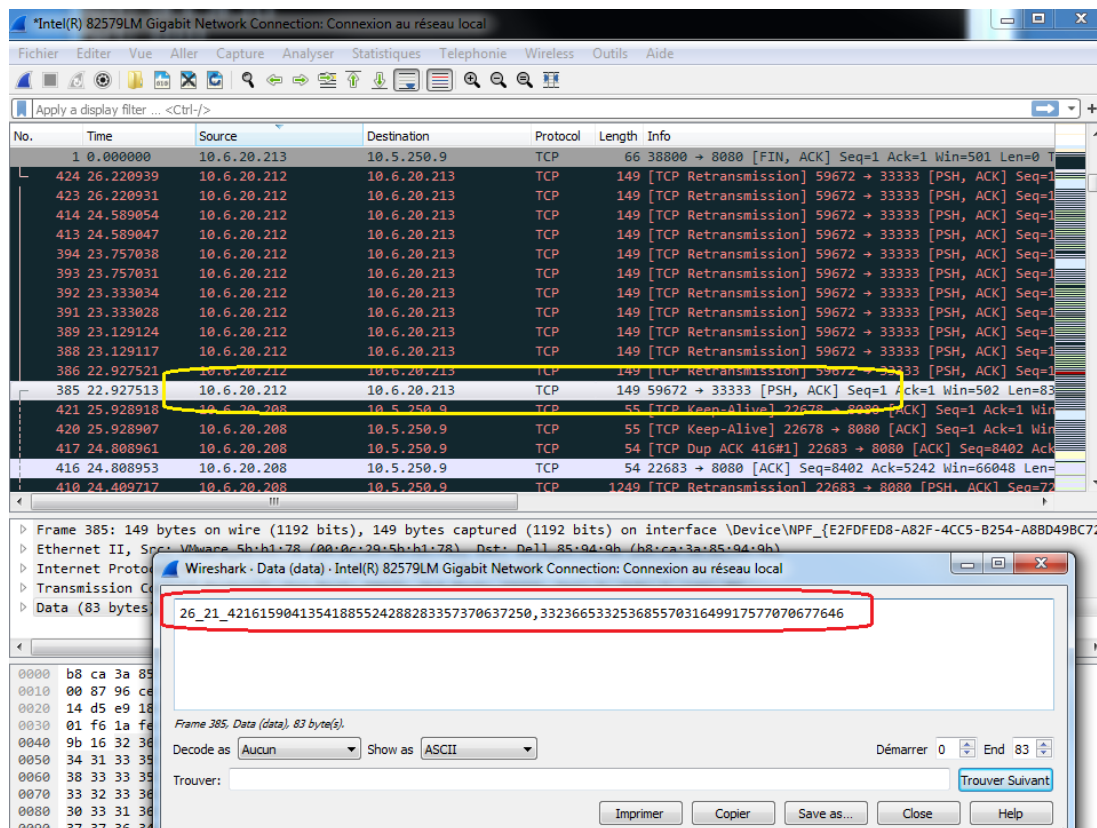
La figure suivante montre l'utilisation de **ettercap** sur un réseau 10.6.20.0, on a détecté deux machines victimes qui sont : 10.6.20.213 et 10.6.20.212.

Ettercap

Ettercap est un outil de sécurité réseau gratuit et open source utilisé dans les attaques man-in-the-middle sur un réseau LAN. Il peut être utilisé pour l'analyse de protocole de réseau informatique et l'audit de sécurité. Il s'exécute sur divers systèmes d'exploitation de type Unix, notamment Linux, Mac OS X, BSD et Solaris, et sur Microsoft Windows. Il est capable d'intercepter le trafic sur un segment de réseau, de capturer des mots de passe et de mener des écoutes actives contre un certain nombre de protocoles courants.



Une fois le cache ARP empoisonné, on doit lancer **Wireshark** avant de commencer la communication entre le client et le serveur. Une fois Wireshark exécuté, on peut commencer la discussion secrète et on aura une fenêtre qui ressemble à peu près à ça :



Le message intercepté, sera mis dans un fichier texte (**encrypted_file.txt**) qui sera traité une fois la clé privée retrouvée. Pour la reconstruction de la clé privée, on doit passer par quatre

étapes : le téléchargement de la clé publique (connue par tout le monde), l'extraction du modulus et de l'exposant de chiffrement, la factorisation du modulus et le calcul de l'exposant de déchiffrement.

Une fois la clé publique récupérée, on récupère le modulus pour le factoriser afin de retrouver les facteurs premiers p et q , la figure suivante montre le processus de factorisation en utilisant le crible quadratique (SIQS), l'opération se termine en seulement 11 secondes vu la taille modeste du modulus utilisé dans cet exemple.

```

Administrator : cmd
1- Trial division to find very small factors
2- Brent's version of the Pollard rho for slightly larger factors
3- Self-Initializing Quadratic Sieve (SIQS) for large factors
Factorising 158967472519671613119496033506350552689 (39 digits)...
--> Trial division and initialising small primes...
--> Primes initialised.
--> No small factors found.
--> Attempting quick Pollard rho (Brent's variant) to find slightly la
rs...
--> No (more) small factors found.
--> Checking whether 158967472519671613119496033506350552689 is a perf
..
--> Not a perfect power.
--> Using Self-Initializing Quadratic Sieve to factorise 1589674725196
6033506350552689 (39 digits)...
----- Step 1/2: Finding smooth relations -----
--> Target: 525 relations
--> Total 24/525 relations.
--> Total 65/525 relations.
--> Total 88/525 relations.
--> Total 120/525 relations.
--> Total 149/525 relations.
--> Total 182/525 relations.
--> Total 218/525 relations.
--> Total 256/525 relations.
--> Total 288/525 relations.
--> Total 310/525 relations.
--> Total 337/525 relations.
--> Total 367/525 relations.
--> Total 398/525 relations.
--> Total 425/525 relations.
--> Total 459/525 relations.
--> Total 500/525 relations.
--> Total 525/525 relations.
----- Step 2/2: Linear Algebra -----
--> Building matrix for linear algebra step...
--> Finding perfect squares using matrix...
--> Finding factors from perfect squares...
--> SIQS: Prime factor found: 10417037966006109733
--> Prime factor found: 10417037966006109733
--> Prime factor found: 15260333411323805533
Success. Prime factors: [10417037966006109733, 15260333411323805533]
Time taken: 11.10011s

```

Une fois les valeurs de p et q trouvées, on peut maintenant calculer la valeur de l'exposant de déchiffrement d en utilisant le script **modinv.py** qui utilise l'inverse modulaire. Comme le montre la figure suivante, on doit copier les valeurs de p , q et e dans leurs emplacements respectifs dans le script avant son utilisation.

```

client.py
1 # This script
2 # YAHIAOUI
3
4 p=10417037
5 q=15260333
6 e = 666766
7
8 n = p*q
9 phi = (p-1)
10
11 def egcd(a, b):
12     if a == 0:
13         return b, 0, 1
14     g, y, x = egcd(b % a, a)
15     return g, x, y - (b // a) * x
16
17 def modinv(e, phi):
    
```

```

C:\py\RSAencryption>modinv.py
P = 10417037966006109733
Q = 15260333411323805533
N = 158967472519671613119496033506350552689
Phi = 158967472519671613093818662129020637424
e = 666766235
The exponent d is found:
d = 3834254766286284616552930873096568387
    
```

Une fois l'exposant de déchiffrement calculé, on peut reconstruire la clé privée comme dans on gardant la même syntaxe de la clé publique et en remplaçant jute l'exposant de chiffrement par l'exposant de déchiffrement :

```

1 128,158967472519671613119496033506350552689,3834254766286284616552930873096568387
    
```

Maintenant, il ne reste qu'à décrypter le message capturé par la clé privée reconstruite en utilisant le script **decrypt_message.py**, ce script utilise le fichier texte **encrypted_file.txt** qui doit se retrouver dans le même dossier:

```

C:\py\RSAencryption>decrypt_message.py
Reading from encrypted_file.txt and decrypting...
Decrypted text:
stop the war peace for all
    
```

Le message est déchiffré avec succès, on peut constater alors le succès du processus d'interception et de cryptanalyse.